

MÓDULO DE 20 MINUTOS · LIVE DEMO DE 9:15

Lean en VS Code, sin improvisar

Guía del facilitador para producir un verde, leer un error rojo, verificar un contraejemplo y reparar el contrato frente a una audiencia que nunca ha visto Lean.

Objetivo: que la audiencia distinga una prueba rechazada, una afirmación refutada y una afirmación reparada; y que Kristian pueda recuperarse de cualquier fallo en menos de veinte segundos.

Kristian López Vargas · Curso de IA generativa para la investigación

Antes de entrar al salón

Abra la carpeta padre `lean_verification_demo/` en VS Code, no solamente `live_demo/`. Así la extensión encuentra `lean-toolchain`, `lakefile.toml` y la versión fijada de Mathlib (v4.31.0). En la terminal integrada ejecute:

```
lake exe cache get
./live_demo/preparar.sh 00
./live_demo/verificar_live_demo.sh
```

Abra `live_demo/DemoEnVivo.lean`, aumente la fuente y espere a que desaparezca el indicador de procesamiento. Cierre paneles y notificaciones que no usará. No actualice Lean ni descargue dependencias durante la sesión.

El archivo empieza verde. Los estados `00`, `10`, `15`, `20`, `30` y `40` son una red de seguridad. En cualquier momento puede ejecutar, por ejemplo, `./live_demo/preparar.sh 30`; VS Code recargará el estado reparado.

0:00-1:15 — Leer un contrato verde

Mostrar: `DemoEnVivo.lean` y la marca verde de la extensión.

Decir:

Lean esta leyendo una oración: para cualesquiera números reales x , y y c , si $x < y$, sumar el mismo c conserva el orden. El enunciado es el contrato; lo que viene después de `by` es la prueba candidata.

Señale `h : x < y`, luego la conclusión. No explique toda la sintaxis. `linarith` busca una prueba de aritmética lineal; el núcleo de Lean comprueba el resultado que produjo.

Mensaje: verde significa «la conclusión se deriva de los supuestos escritos», no «los supuestos describen el mundo».

1:15-3:15 — Producir un rojo comprensible

Pregunte y reciba una votación rápida:

A tiene más casos que B y también más población. ¿Se sigue siempre que A tiene mayor incidencia?

Pegue debajo del comentario final:

```
theorem mas_casos_implica_mayor_tasa
  (casosA casosB pobA pobB : ℝ)
  (hA : 0 < pobA) (hB : 0 < pobB)
  (hcasos : casosB < casosA)
  (hpob : pobB < pobA) :
  casosB / pobB < casosA / pobA := by
  exact hcasos
```

Espere el subrayado rojo. En Infoview, lea únicamente las dos expresiones:

- ofrecido: $\text{casosB} < \text{casosA}$;
- esperado: $\text{casosB} / \text{pobB} < \text{casosA} / \text{pobA}$.

Decir:

Yo ofrecí una comparación de conteos como si fuera una comparación de tasas. Lean rechazó ese paso preciso. Todavía no ha demostrado que el teorema sea falso: quizá simplemente ofrecí una mala prueba.

Si el pegado falla o el mensaje no aparece en 10 segundos, ejecute
`./live_demo/preparar.sh 10`.

3:15-3:55 — Segundo rojo: búsqueda fallida

Cambie únicamente la última línea de `exact hcasos` a:

```
linarith
```

Lean 4.31.0 muestra el diagnóstico observado:

```
linarith failed to find a contradiction
```

Decir:

Son dos rojos distintos. Antes ofrecimos un objeto del tipo incorrecto. Ahora `linarith` sí buscó una prueba dentro de su procedimiento y no la encontró. Una búsqueda fallida todavía no prueba que el enunciado sea falso.

El contraste que la sala debe conservar es:

- `Type mismatch`: la prueba ofrecida no tiene el tipo prometido;
- `linarith failed...`: esta táctica no encontró una prueba;
- `contraejemplo verde`: Lean comprobó valores que refutan el «siempre».

El texto literal está fijado al `lean-toolchain` v4.31.0. Fallback: `./live_demo/preparar.sh 15`.

Use `Cmd+Z` dos veces como máximo para retirar el intento y el bloque pegado. Si el archivo no vuelve a verde después del segundo `Cmd+Z`, no depure en vivo: ejecute `./live_demo/preparar.sh 00`. Si prefiere no volver a pegar, salte a `./live_demo/preparar.sh 20` y narre el contraejemplo ya congelado.

3:55-5:15 — Convertir la sospecha en contraejemplo

Pegue:

```
theorem contraejemplo_mas_casos_menor_tasa :  
  (1 : ℝ) < 2 ∧  
  (10 : ℝ) < 100 ∧  
  ¬ ((1 : ℝ) / 10 < (2 : ℝ) / 100) := by  
  norm_num
```

Decir:

A tiene 2 casos entre 100 personas: 2 %. B tiene 1 entre 10: 10 %. Se cumplen las dos comparaciones iniciales, pero la comparación de tasas va al revés. Ahora Lean sí acepta una refutación concreta. Un caso basta contra «siempre».

`norm_num` resuelve aquí cuentas numéricas exactas; no es una simulación ni una aproximación decimal.

Fallback: `./live_demo/preparar.sh 20`.

5:15-6:45 — Reparar el contrato, no maquillar la prueba

Pegue:

```
theorem misma_poblacion_mayor_tasa
  (casosA casosB poblacion : ℝ)
  (hpob : 0 < poblacion)
  (hcasos : casosB < casosA) :
  casosB / poblacion < casosA / poblacion := by
  gcongr
```

Decir:

No obligamos a Lean a aceptar el teorema anterior. Escribimos otro contrato: con el mismo denominador positivo, más casos sí implica una tasa mayor. El supuesto visible hace el trabajo matemático y abre la pregunta sustantiva: ¿es legítimo tratar estas poblaciones como comparables?

`gcongr` construye la prueba de la desigualdad; Lean comprueba esa prueba. Fallback:
`./live_demo/preparar.sh 30`.

6:45-8:45 — Cerrar con una mezcla

Pegue:

```
theorem mezcla_entre_extremos (x y a : ℝ)
  (hxy : x ≤ y) (ha0 : 0 ≤ a) (ha1 : a ≤ 1) :
  x ≤ a * x + (1 - a) * y ∧
  a * x + (1 - a) * y ≤ y := by
  constructor <=> nlinarith
```

Decir:

Si a es un peso entre cero y uno, la mezcla queda entre los extremos. Si quitamos esas cotas, ya no hablamos necesariamente de una mezcla: por ejemplo, con $x = 0$, $y = 10$ y $a = 2$, obtenemos -10 , una extrapolación.

Conexión honesta:

Disentangling formaliza objetos mucho más ricos: loterías finitas, pesos válidos y valor esperado. Este teorema no reemplaza esa formalización. Es el pequeño patrón algebraico que explica por qué las cotas del peso importan.

Fallback: `./live_demo/preparar.sh 40`.

8:45-9:15 — La frase de salida

Un modelo puede ayudarme a escribir y corregir Lean. Lean puede certificar que una conclusión sigue del contrato escrito. Ninguno decide por mí si el contrato traduce bien la pregunta ni si sus supuestos describen mis datos.

Termine en verde. No improvise otro teorema después de esta frase.

Recuperación rápida

Situación	Acción visible	Qué decir
La extensión sigue procesando	espere 10 s; luego cambie al estado preparado	«La primera carga es infraestructura, no razonamiento.»
Infoview quedó oculto	paleta de comandos → Lean 4: Infoview: Toggle	«Recupero la ventana donde Lean explica el estado.»
El servidor no responde	paleta → Lean 4: Restart Server ; si persiste, use snapshot	«Reinicio el verificador, no cambio el teorema.»
El pegado quedó incompleto	<code>./live_demo/preparar.sh</code> con el número del paso	«Vuelvo a un estado reproducible.»
Dos Cmd+Z no devuelven verde	deje de deshacer y salte al snapshot <code>00</code> o al siguiente	«No depuramos mecanografía frente a la sala.»
Un rojo no coincide	abra <code>estados/10_rojo.lean</code> o <code>estados/15_busqueda_falla.lean</code>	«Este es el mismo paso, congelado antes de clase.»
Quedan menos de 4 minutos	salte a <code>20</code> , narre el contraejemplo y luego <code>40</code>	conservar contraejemplo + cierre; omita tecleo de reparación
Lean o VS Code falla por completo	muestre los estados ya abiertos, sin fingir ejecución	explique qué salida se verificó antes de clase

Qué no afirmar

- Un error de táctica no demuestra por sí solo que un teorema sea falso.
- Lean no valida los datos ni la interpretación de «incidencia».
- $pobA < pobB$ o el orden de poblaciones no basta para ordenar cocientes.
- El teorema de mezcla no es la formalización completa de Disentangling.
- No diga que la IA «probó» algo si solo produjo código que todavía está rojo.

Cabina de mando · una página

Abra la carpeta padre en VS Code, prepare el estado 00 y espere a que Mathlib termine de cargar antes de entrar al minuto 0.

Antes de clase

```
lake exe cache get
./live_demo/verificar_live_demo.sh
./live_demo/preparar.sh 00
```

Reloj del demo

0:00	Verde	Leer contrato; señalar supuesto y conclusión
1:15	Rojo	Pegar trampa; leer ofrecido vs. esperado
3:15	Rojo 2	linarith busca y falla; todavía no hay refutación
3:55	Contraejemplo	2/100 frente a 1/10; un caso refuta siempre
5:15	Reparación	Mismo denominador positivo; contrato nuevo
6:45	Mezcla	Peso entre 0 y 1; extrapolación si $a = 2$
8:45	Cierre	Lean verifica el paso, no el mundo

Botones de recuperación

```
./live_demo/preparar.sh 10 # rojo esperado
./live_demo/preparar.sh 15 # búsqueda fallida
./live_demo/preparar.sh 20 # contraejemplo
./live_demo/preparar.sh 30 # reparación
./live_demo/preparar.sh 40 # mezcla
```

Si quedan menos de cuatro minutos: salte a 20, narre el contraejemplo y cierre con 40.
Si VS Code falla por completo: muestre los estados congelados y diga que fueron verificados antes de clase.